

# Bismillah ir-Rahman ir-Rahim

## FATIMA - Anatomy of the Code (End of Day)

**Artefact:** fatima-core @ commit dfb6205, package 0.1.0 **Extraction method:** parsed from the live AST - every symbol below exists in the code exactly as written. **Annotations:** [P1]-[P5] property served - [L1]-[L5] verification level - † canon finding (see FATIMA Canon Rev. 2) - ♦ module verdict **Date:** 2026-10-07 **Canon findings closed today:** F-03, F-04, F-01, F-05 (4 of 8) **Test suite:** 34/34 passing (0.20s)

```
fatima-core/
├── pyproject.toml                                # package metadata, version 0.1.0
├── FATIMA_paper.fatima                          # the paper, encoded in its own format
├── Ghayb_l1C_paper.fatima                      # second encoded document
├── CODE_EVOLUTION.md                           # Era I--VII historical record (Rev 1)
├── CODE_EVOLUTION_CANON_V2.md                  # Code Evolution Canon Rev 2
├── COMPLETE_CODE_AUDIT.md                     # companion audit
├── AL_FURQAN_ERRATA_SECOND_EDITION.md/.pdf    # Round III council errata (C-01..C-12)
├── FATIMA_ANATOMY_V2.md/.pdf                  # evolved anatomy
├── FATIMA_PBN_V2.md/.pdf                      # evolved physiology/biochemistry/neurochemistry
├── FATIMA_SCHEMATICS_V2.md/.pdf              # evolved code schematics
├── fatima/                                     # 5,009 LOC of Python, 33 modules
│   ├── __init__.py                            [25] # P1--P5 manifesto, __version__
│   └── cli.py                                [347] # entry point --
♦ RATIFIED (not yet provenance-aware)
    ├── core/                                  # the body
    │   ├── __init__.py                        [19] # exports the five organs
    │   ├── atom.py                          [181] # the cell ♦ RATIFIED †F-03 CLOSED
    │   ├── bond.py                         [171] # the ligament ♦ RATIFIED
    │   ├── encoding.py                     [528] # the blood [P1] ♦ RATIFIED †F-07,F-08
    │   ├── molecule.py                    [450] # the organ [P3] ♦ RATIFIED †F-03 CLOSED
    │   └── composite.py                    [324] # the skeleton ♦ RATIFIED
    ├── graft/                                # the immune system (NEW --
    │   └── graft_protocol)
    │       ├── __init__.py                  [16] # graft protocol exports
    │       └── manifest.py                  [207] # graft lifecycle ♦ RATIFIED (evidence-
status)
    │   ├── report.py                       [88] # graft-layer report ♦ RATIFIED
    │   ├── independence.py                 [100] # independence ledger ♦ RATIFIED (nominal, not effective)
    │   ├── anchor.py                      [117] # external anchor ♦ RATIFIED (protocol, not proof)
    │   ├── composite.py                   [98] # composite boundary ♦ RATIFIED
    │   ├── adversarial/                   # adversarial regression memory
    │   │   ├── __init__.py                [8] # fixture exports
    │   └── fixtures.py                    [121] # fixture pairs ♦ RATIFIED (plumbing, not semantics)
    │   ├── verification/
    │   │   ├── __init__.py                [9] # the senses
    │   │   └── verdict.py                 [130] # L1--L5 map
    │   └── bearing)
    │       ├── syntactic.py                [23] # [L1] sight ♦ RATIFIED †F-02
    │       ├── structural.py               [28] # [L2] touch ♦ RATIFIED
    │       └── holographic.py              [174] # [L3] memory ♦ RATIFIED
```

```

04 |   | semantic.py           [232] # [L4/L5] judgement ♦ RATIFIED †F-
   |   | provenance.py       [198] # [L5] external trust ♦ RATIFIED (NEW) †F-
05 |   | composite.py         [319] # composite senses ♦ RATIFIED
   |   | properties/         # the five claims, stated as predicates
   |   |   | __init__.py      [3]
   |   |   | holographic.py   [37] # [P1] ♦ RATIFIED
   |   |   | fitrah.py        [35] # [P2] ♦ RATIFIED (heuristic, honestly)
   |   |   | tawhidic.py      [134] # [P3] ♦ RATIFIED
   |   |   | meaning.py       [125] # [P4] ♦ PROMOTED †F-
03 |   | authenticity.py     [57] # [P5] ♦ PROMOTED †F-
01 |   | formats/            # the mouth and hands
   |   |   | __init__.py      [3]
   |   |   | document.py      [547] # text -> Molecule ♦ RATIFIED †F-06 lexical
   |   |   | serialization.py [155] # .fatima JSON I/O ♦ RATIFIED

```

**core - the body**

```
fatima/core/atom.py [181 LOC] the cell

class Atom @dataclass
  | atom_id: str identity within the molecule
  | content: str the meaning it carries
  | content_type: str one of _VALID_TYPES = {proposition, definition,
  | evidence, claim, reference, metadata, invocation}
  | content_hash: str SHA-256(content) -- [L1] marker
  | semantic_hash: str SHA-256(content|bond_ids|shard) --
[P4] marker †F-03 CLOSED
  | bond_ids: list[str] every bond it participates in
  | holographic_shard: bytes its k-of-n share of the bond graph
  | shard_threshold: int the k of the scheme
  | position: int document order (metadata, not meaning)

  def __post_init__(self) validate type; hash content
  def __compute_content_hash(self) -> str [L1]
  def semantic_hash_value(self) -> str [GRAFTED F-03] pure computation --
  returns SHA-256 without mutation;
  verification-safe form
  def compute_semantic_hash(self) -> str [GRAFTED F-03] BUILD-TIME only --
  writes self.semantic_hash; calls
  semantic_hash_value() then stores
  def verify_syntactic(self) -> bool † F-02: circular -- stored hash vs.
  recomputed hash, same untrusted file

  @property has_shard -> bool
  @property structural_weight -> int len(bond_ids); NC-
P2 loss weighting
  def to_dict / from_dict serialisation
  def __repr__

fatima/core/bond.py [171 LOC] the typed relationship
```

```

class BondType(enum.Enum)
├── DEFINES ┌ structural      reciprocal: DEPENDS_ON
├── DEPENDS_ON └ (load-bearing) reciprocal: DEFINES
├── IMPLIES ┌ reciprocal: DEPENDS_ON
├── CONTRADICTS └ reciprocal: CONTRADICTS (symmetric)
├── SUPPORTS ┌ non-structural no automatic reciprocal
├── REFERENCES └ (enriching)
├── ELABORATES
├── QUALIFIES
└── @property is_structural -> bool [P3] maps to Tawhidic Unity

class Bond @dataclass(frozen=True) -- immutable
├── source_id, target_id: str direction matters
├── bond_type: BondType
├── weight: float (0.0, 1.0], enforced in __post_init__
├── rationale: str the NC-P1 audit trail -- must survive disclosure
├── def __post_init__ reject self-bonds and out-of-range weights
├── @property bond_id -> str SHA-256(src:tgt:type)[:16] -- one bond per
├── (src,tgt,type) triple, by construction
├── def reciprocal_type() -> Optional[BondType]
├── def to_dict / from_dict

fatima/core/encoding.py [528 LOC] the field and the shares [P1]

├── GF(2^8) ARITHMETIC -- AES polynomial 0x11B, generator 3
├── _GF_EXP[512], _GF_LOG[256] log/anti-log tables, built by _init_gf_tables()
├── _GF_MUL_TABLE: np.uint8[256,256] 256 KB lookup -- vectorised multiply, fits L2 cache
├── _gf_mul_raw(a,b) shift-and-XOR, table bootstrap only
├── gf_mul gf_div gf_pow gf_add gf_add = XOR (†gf_pow: dead code)

├── SHAMIR MACHINERY
├── _make_polynomial(secret_byte, degree) random nonzero coeffs via os.urandom
├── _eval_polynomial(coeffs, x) Horner's method over GF(2^8)
├── _lagrange_interpolate(points) scalar reference path -- dead code
├── _compute_lagrange_basis(x_coords) basis coefficients L_i(0), computed once
├── _vectorised_lagrange_decode(matrix, basis)
├── secret[pos] = XOR_i( GF_MUL[basis[i], shard[i,pos]] )
├── table-index + XOR-reduce; no Python loops over bytes

├── class HolographicParams @dataclass
├── ┌ threshold: int / total: int
├── ┌ @property redundancy_ratio
├── ┌ @classmethod for_document(n_atoms, ratio=0.5)
├── ┌ k = max(2, ceil(n*ratio)); hard limit n <= 255 (field order - 1)

├── encode_holographic(bond_graph_json, atom_ids, threshold)
├── ┌ zlib-compress(level=9) -> per byte, fresh degree-(k-1) polynomial
├── ┌ -> evaluate at points 1..n -> one shard per atom
├── ┌ † F-07: Python-level 0(bytes * n) loop -- 3.84 s @ 122 atoms (measured)

├── decode_holographic(shards, atom_ids_used, threshold, all_atom_ids)
├── ┌ needs >= k shards; points recovered from position map; vectorised

```

- `apply_holographic_encoding(molecule, ratio=0.5)` in place; serialises the bond graph as the "secret"; writes (shard, threshold) into every atom
- `verify_holographic_reconstruction(molecule, subset_ids=None)`  
     decode -> compare JSON to live graph  
     † F-08: corruption surfaces as zlib exception, not as a localised verdict

fatima/core/molecule.py [450 LOC] the document as graph [P3]

```
class Molecule @dataclass
├── molecule_id: str
├── title: str
├── atoms: dict[str, Atom]
├── bonds: dict[str, Bond]
├── metadata: dict
├── bond_graph_hash: str the structural fingerprint

— SURGERY
  add_atom (uniqueness enforced) -- get_atom -- remove_atom (dangles become
  detectable -- loss is survivable by design) -- add_bond (endpoints must exist,
  registers bond_id into both atoms) -- _invalidate_graph_hash

— GRAPH READING
  get_bonds_for / get_outgoing_bonds / get_incoming_bonds
  adjacency -- is_connected -- connected_components -- dangling_bonds
  missing_reciprocals -- structural cross-reference audit
  atom_count -- bond_count -- structural_bond_count

— FINGERPRINTING [GRAFTED F-03]
  bond_graph_hash_value() pure computation -- returns SHA-256 of sorted
                           (src:tgt:type:weight) lines without mutation;
                           verification-safe form
  compute_bond_graph_hash() BUILD-TIME only -- writes self.bond_graph_hash;
                           calls bond_graph_hash_value() then stores
  compute_all_semantic_hashes() per-atom compute_semantic_hash() then
                              compute_bond_graph_hash()

— verify_structural() -> VerificationReport [L2]
  dangling | connectivity (Tawhidic Unity) | reciprocals | bond_id consistency
  empty molecule -> UNKNOWN, not VERIFIED

— verify_syntactic() -> VerificationReport [L1] one hash check per atom

— atoms_by_position -- __iter__ -- to_dict/from_dict -- to_json/from_json
```

fatima/core/composite.py [324 LOC] beyond the field limit

```
— MAX_ATOMS_PER_SUB = 200 headroom below the 255-point GF ceiling

class InterMoleculeBond cross-section bond; verified at composite level
├── bond_id = SHA-256(src_mol:src_atom:tgt_mol:tgt_atom:type)[:16]

class CompositeMolecule @dataclass
├── sub_molecules: dict[str, Molecule]
├── inter_bonds: dict[str, InterMoleculeBond]
```

```

├─ molecule_order: list[str]          document order
├─ composite_hash: str                SHA-256 over all sub hashes + inter-bonds
├─ add_sub_molecule -- add_inter_bond (four-way existence validation)
├─ apply_holographic_encoding(ratio)  each sub-molecule encoded independently
├─ finalise()                        semantic hashes everywhere, then composite hash
├─ is_connected()                    composite-level Tawhidic Unity
├─ total_atoms -- total_bonds -- sub_molecule_count
├─ to_dict / from_dict

```

---

## verification - the senses

fatima/verification/verdict.py [130 LOC] the conscience

```

class Verdict(enum.Enum)          VIOLATED < UNKNOWN < VERIFIED
├─ __and__ lattice MEET -- composition can only degrade (NC-P2, in 5 lines)
├─ __or__  lattice JOIN -- any sufficient cross-section suffices (P1)

@dataclass(frozen=True) class VerificationReport
├─ verdict -- level (1..5, enforced) -- confidence (0..1, enforced)
├─ examined -- total -- violations: tuple -- level_name
├─ @staticmethod compose(*reports) verdict = meet; confidence = min;
                                   violations = concat; empty -> UNKNOWN

```

The strongest module in the codebase. Canon C-2.

```

syntactic.py [23] [L1] verify_syntactic(molecule) -> Molecule.verify_syntactic()
structural.py [28] [L2] verify_structural(molecule) -> Molecule.verify_structural()
               (canonical entry points; thin by design)

```

fatima/verification/holographic.py [174 LOC] [L3] the memory

```

├─ verify_holographic(molecule, sample_size=20, seed=None)
├─   └─ n < 2 -> VERIFIED(trivial) / UNKNOWN(empty)
├─   └─ no shards -> UNKNOWN ("encoding not applied")
├─   └─ shards < k -> UNKNOWN, confidence = held/n (NC-
P2: not VIOLATED)
├─   └─ full reconstruction fails -> VIOLATED immediately
├─   └─ else sample k-subsets
├─       C(n,k) <= sample_size -> exhaustive via itertools.combinations
├─       C(n,k) > sample_size -> rng.sample(seed=able) -- Era IV harness discipline
├─       all pass -> VERIFIED -- none or partial -> VIOLATED

```

fatima/verification/semantic.py [232 LOC] [L4/L5] judgement [GRAFTED]

```

├─ verify_fitrah_alignment(molecule) [L4] --
heuristics, honestly labelled
1. claims with no evidence AND no propositions -> "possible selective emphasis"
2. zero structural bonds -> "no skeleton"
3. claims with no incoming SUPPORTS/IMPLIES -> unsupported
4. definitions without ELABORATES either way -> unelaborated
5. any CONTRADICTS bond -> flagged (may be deliberate dialectic)
   violations -> UNKNOWN (not VIOLATED -- incompleteness != corruption)

```

```

└ verify_authenticity(molecule, provenance?, key_provider?)      [L5] --
evolved architecture
  [GRAFTED F-04/F-01/F-05]
  Step 1: L1-L3 unchanged (syntactic, structural, holographic)
  Step 2: L4 = compose(check_meaning_integrity, verify_fitrah_alignment)
           †F-04 CLOSED: check_meaning_integrity() now wired into pipeline
  Step 3: internal = compose(L1, L2, L3, L4)
  Step 4: L5 = verify_provenance(envelope, expected_commitment, key_provider)
           †F-01 CLOSED: self-authentication eliminated -- no envelope -> UNKNOWN
           †F-05 CLOSED: external anchor + author signature + reviewer attestation
  Step 5: final = meet(internal, L5_provenance)
           A document cannot reach VERIFIED without external evidence.

```

fatima/verification/provenance.py [198 LOC] [L5] external trust (NEW)

```

@dataclass(frozen=True) class ProvenanceEnvelope
├ author_key_id: str          signing key identifier
├ signature: bytes           Ed25519 over commitment_hash
├ commitment_hash: str       must match molecule.bond_graph_hash
├ anchor_provider: str       "rfc3161", "opentimestamps", "null"
├ anchor_proof: bytes        provider-specific proof
├ anchor_timestamp: str      ISO-8601
├ reviewer_key_id: str|None   independent reviewer
├ reviewer_attestation: bytes|None

class PublicKeyProvider(Protocol)
└ def get_public_key(key_id: str) -> object|None

class NullKeyProvider
└ always returns None -- honest default when no trust store configured

def verify_provenance(envelope, expected_commitment, key_provider)
├ no envelope -> UNKNOWN ("cannot establish without external evidence")
├ check 1: commitment_hash matches expected -> VIOLATED on mismatch
├ check 2: author signature verifies -> UNKNOWN if key absent
├ check 3: anchor_provider != "null" -> UNKNOWN if null
├ check 4: reviewer_key_id != author_key_id -> VIOLATED on collapse

```

fatima/verification/composite.py [319 LOC] composite senses

```

└ verify_composite(composite, holographic_sample_size=20, holographic_seed=None)
  each sub-molecule: L1, L2, L3(sampled, seeded), L4 -> composed L5
  composite-level: connectivity -- cross-section fitrah -- composite hash
  overall = meet of every sub-L5 and every composite check
├ _check_composite_connectivity    <=1 sub -> trivially VERIFIED
├ _check_cross_section_fitrah      endpoint existence, weight ranges
├ _check_composite_hash            unset -> UNKNOWN; mismatch -> VIOLATED
└ class CompositeVerificationResult
  sub_reports -- composite_reports --
overall
  .summary()                      human-readable verdict tree

```

## properties - the five claims, as predicates

```
fatima/properties/                                     each: check_X(molecule) -
> VerificationReport                                  plus is_X(molecule) -> bool

— holographic.py [37] P1 wraps L3 sampling                ♦ true
— fitrah.py      [35] P2 wraps L4 heuristics              ♦ true-as-heuristic
— tawhidic.py    [134] P3 connectivity -- dangling -- isolation --
♦ true of the graph
      removal-detectability -- structural
      density floor (n/4)                † F-02 caveat:
      any violation -> VIOLATED            adversary, vs. accident

— meaning.py     [125] P4 ♦ PROMOTED from x † F-03 CLOSED
  [GRAFTED F-03 -- GRAFT-F03-PURITY-001]
  check 1: bond-graph-hash staleness -- calls bond_graph_hash_value() (pure),
    compares to stored bond_graph_hash -- mismatch -> VIOLATED,
    absent -> UNKNOWN
  check 2: per-atom semantic commitment -- calls atom.semantic_hash_value() (pure),
    compares to stored atom.semantic_hash -- tamper detectable
  check 3: rationale coverage -- the NC-P1 disclosure check
  Purity contract: verification NEVER writes; missing evidence -> UNKNOWN

— authenticity.py [57] P5 ♦ PROMOTED from x † F-01 CLOSED
  [GRAFTED F-01/F-05 -- provenance authenticity]
  check_authenticity(molecule, provenance?, key_provider?)
    -> delegates to verify_authenticity (semantic.py)
  is_authentic(molecule, provenance?, key_provider?)
    -> True only when verdict == VERIFIED
  Without provenance envelope, always returns UNKNOWN/False.
  Self-authentication eliminated; external evidence required.
```

---

## graft - the immune system (NEW)

```
fatima/graft/manifest.py                               [207 LOC] graft lifecycle

class GraftStatus(enum.Enum)
— PROPOSED      declared, not yet applied
— APPLIED       mechanism replaced on substrate
— UNKNOWN       applied but not independently reviewed
— VERIFIED      applied and independently attested
— REVERTED      rolled back; recorded permanently

@dataclass(frozen=True) class Graft
— graft_id: str      unique identifier
— target_path: str   file in fatima-core
— old_symbol: str    symbol being replaced
— new_module: str    v0.2 module supplying replacement
— rationale: str     Canon finding description
— canon_ref: str     F-01 through F-08 or other
— status: GraftStatus lifecycle position
```

```

└─ reviewer_attestation: bytes|None    Ed25519 over (graft_id, status)
└─ reviewer_key_id: str|None          reviewer key identifier
└─ def canonical_bytes() -> bytes      deterministic serialisation for signatures

def compute_graft_verdict(graft, author_key_id, reviewer_public_keys?)
    PROPOSED -> absent; REVERTED -> invalid
    no reviewer attestation -> absent; reviewer == author -> absent
    reviewer key not in trust store -> absent
    signature verification -> ok / invalid
    status != VERIFIED -> absent (signed but not promoted)

@dataclass(frozen=True) class GraftManifest
└─ release_id: str
└─ grafts: tuple[Graft, ...]
└─ def compute(author_key_id, reviewer_public_keys?)
    aggregate = meet of all graft verdicts
    any UNKNOWN graft -> manifest cannot be VERIFIED

fatima/graft/report.py [88 LOC] graft-layer report

@dataclass(frozen=True) class GraftReport
└─ layer: str                "graft", "independence", "anchor", "composite"
└─ verdict: Verdict          VERIFIED / UNKNOWN / VIOLATED
└─ confidence: float          0.0--1.0
└─ notes: tuple[str, ...]    human-readable details
└─ code: str                  machine-readable tag

@classmethod absent(layer, reason)    evidence absent -> UNKNOWN
@classmethod invalid(layer, reason)    evidence invalid -> VIOLATED
@classmethod ok(layer, confidence, note) evidence valid -> VERIFIED
def meet(self, other)                  lattice meet -- monotone degradation

fatima/graft/independence.py [100 LOC] structural non-
collusion

@dataclass(frozen=True) class EvidenceParty
└─ role: str                  "author", "semantic_reviewer", or "anchor"
└─ key_id: str                 cryptographic key identifier
└─ lineage: str                declared organisational lineage (soft signal)

@dataclass(frozen=True) class IndependenceRecord
└─ author: EvidenceParty
└─ semantic_reviewer: EvidenceParty|None
└─ anchor: EvidenceParty|None

def check_independence(record) -> GraftReport
└─ condition 1: reviewer key != author key -> VIOLATED on collapse
└─ condition 2: anchor key != author AND != reviewer -> VIOLATED
└─ condition 3: reviewer lineage != author lineage -> UNKNOWN (soft, not VIOLATED)
    Passing this check is necessary, never sufficient.

fatima/graft/anchor.py [117 LOC] external anchor interface

@dataclass(frozen=True) class AnchorProof
└─ provider: str              "rfc3161", "opentimestamps", "test-double"

```

```

└─ payload: bytes           provider-specific proof bytes
└─ anchored_at: str        ISO-8601 timestamp
└─ commitment_hash: str    the hash the anchor covers

class AnchorProvider(Protocol)
└─ name: str
└─ def verify(proof, expected_hash) -> bool

class NullAnchor           returns False -- honest default
class TestDoubleAnchor     commitment match only -- for tests, not trust

def verify_anchor(proof, provider, expected_commitment_hash) -> GraftReport
└─ no proof                -> absent
└─ null provider           -> absent
└─ wrong commitment       -> invalid
└─ provider rejects       -> invalid

fatima/graft/composite.py [98 LOC] composite boundary

```

```

@dataclass(frozen=True) class CompositeGraftReport
└─ overall: GraftReport
└─ per_sub: dict[str, VerificationReport]
└─ inter_bond_status: GraftReport

def verify_composite_grafts(sub_reports, inter_bond_report)
    composite verdict = meet of all sub-molecule verdicts + inter-bond
    empty sub-map -> UNKNOWN; monotone -- never aggregates upward

fatima/graft/adversarial/fixtures.py [121 LOC] adversarial regression memory

Relation = Literal["paraphrase", "negation", "elaboration"]

@dataclass(frozen=True) class FixturePair
└─ fixture_id: str          unique identifier
└─ source: bytes           original text
└─ graph_bytes: bytes      proposed graph encoding
└─ declared_relation: Relation "paraphrase" | "negation" | "elaboration"
└─ expected_faithful: bool  correct reviewer verdict

PARAPHRASE_PAIRS (2 fixtures) meaning-preserving reformulations
NEGATION_PAIRS (2 fixtures) meaning-inverting reformulations

def check_fixture_pair(pair, reviewer_attested_faithful) -> (bool, str)
    tests plumbing, not semantic understanding -- disclosed as such

```

## formats + cli - the mouth and hands

```

fatima/formats/document.py [547 LOC] text -> molecule

└─ encode_document(text, title, molecule_id, metadata, holographic_ratio=0.5)
    molecule_id defaults to SHA-256(text)[:16]
    <= 200 atoms -> _encode_single_molecule -- else -> _encode_composite

```

```

| PIPELINE (single): atomise -> infer bonds -> holographic encode -> finalise
|
| _atomise / _split_paragraphs / _classify_content
|   "#" -> definition -- "Bismillah" -> invocation -- **Definition -> definition
|   because/therefore/since/thus... -> evidence -- must/should/cannot... -
> claim
|   " | " / "---" -> metadata -- else proposition
|   † F-06: classification is lexical -- paraphrase changes the molecule,
|       keyword-loaded inversion can keep its shape
|
| _infer_bonds(atoms, full_text)
|   R1 adjacency: heading->DEFINES next (+ DEPENDS_ON back); else -
>ELABORATES prev
|   R2 evidence -> SUPPORTS nearest preceding claim (by position distance)
|   R3 invocation -> QUALIFIES every other atom
|   R4 >=2 shared Capitalised terms -> REFERENCES (weight 0.4)
|   dedup by bond_id
|
| COMPOSITE PATH: _split_at_sections -> _split_group_at_subheadings ->
|   _split_evenly -- merge groups < 5 atoms -- _group_title --
|   _add_inter_bonds (sequential ELABORATES 0.7, heading DEPENDS_ON 0.6,
|   invocation QUALIFIES 0.4 across every other sub-molecule)
|
fatima/formats/serialization.py [155 LOC] .fatima I/O
| save_fatima(structure, path, verification_reports, encoding_params)
|   {"fatima_version":"0.1.0","type":"molecule|"composite", ...}
|   verification snapshot: UTC timestamp + per-level reports + composed overall
| load_fatima(path) backward compatible: missing "type" => single molecule
|
fatima/cli.py [347 LOC]
| fatima encode <input> [-o out] [-t title] [--ratio 0.5]
| fatima verify <input> [--sample-size 20] [--seed n]
|   single: L1, L2, L3(sampled), L4, L5 -- exit 0 iff L5 == VERIFIED
|   composite: verify_composite().summary() -- same exit rule
|   (not yet provenance-aware -- CLI does not accept provenance envelopes)
| fatima inspect <input> counts, type/bond distribution, holographic params,
|   stored verification snapshot

```

---

## Test suite - regression contract

tests/ -- 34 tests, 3 files, 0.20s

test\_graft\_f03\_purity.py (4 tests)

```

test_semantic_hash_value_is_pure      compute without mutation
test_compute_semantic_hash_commits    build-time writes
test_meaning_integrity_verified_after_commit full cycle -> VERIFIED
test_meaning_integrity_detects_tampering bond_ids tamper -> VIOLATED

```

test\_graft\_f04\_f01\_f05.py (12 tests)

```

F-04 wiring:
test_f04_meaning_integrity_wired_into_verify_authenticity
test_f04_missing_commitments_degrade_full_pipeline

```

F-01 self-auth elimination:  
 test\_f01\_no\_provenance\_yields\_unknown\_not\_verified  
 test\_f01\_check\_authenticity\_wrapper\_also\_unknown\_without\_provenance  
 test\_f01\_is\_authentic\_false\_without\_provenance  
 F-05 external anchoring:  
 test\_f05\_provenance\_with\_mismatched\_commitment\_is\_violated  
 test\_f05\_provenance\_with\_matching\_commitment\_but\_no\_key\_is\_unknown  
 test\_f05\_null\_anchor\_is\_unknown  
 test\_f05\_self\_attestation\_detected  
 test\_f05\_no\_provenance\_envelope\_is\_unknown  
 Integration:  
 test\_full\_pipeline\_with\_provenance\_envelope\_unknown\_key  
 test\_full\_pipeline\_provenance\_through\_check\_authenticity

test\_graft\_protocol.py (18 tests)

Graft infrastructure: manifest, independence, anchor, composite,  
 adversarial fixtures

## Canon status register

| ID   | Finding                             | Severity | Status | Graft closure          |
|------|-------------------------------------|----------|--------|------------------------|
| F-01 | Authenticity is forgeable           | CRITICAL | CLOSED | F-04/F-01/F-05 coupled |
| F-02 | Syntactic verification is circular  | HIGH     | OPEN   | --                     |
| F-03 | Meaning-integrity is vacuous        | CRITICAL | CLOSED | GRAFT-F03-PURITY-001   |
| F-04 | P4 is orphaned from pipeline        | HIGH     | CLOSED | F-04/F-01/F-05 coupled |
| F-05 | No external anchoring               | HIGH     | CLOSED | F-04/F-01/F-05 coupled |
| F-06 | Bond inference is lexical           | MEDIUM   | OPEN   | --                     |
| F-07 | Encoder is slow                     | MEDIUM   | OPEN   | --                     |
| F-08 | Shard corruption fails by exception | LOW      | OPEN   | --                     |

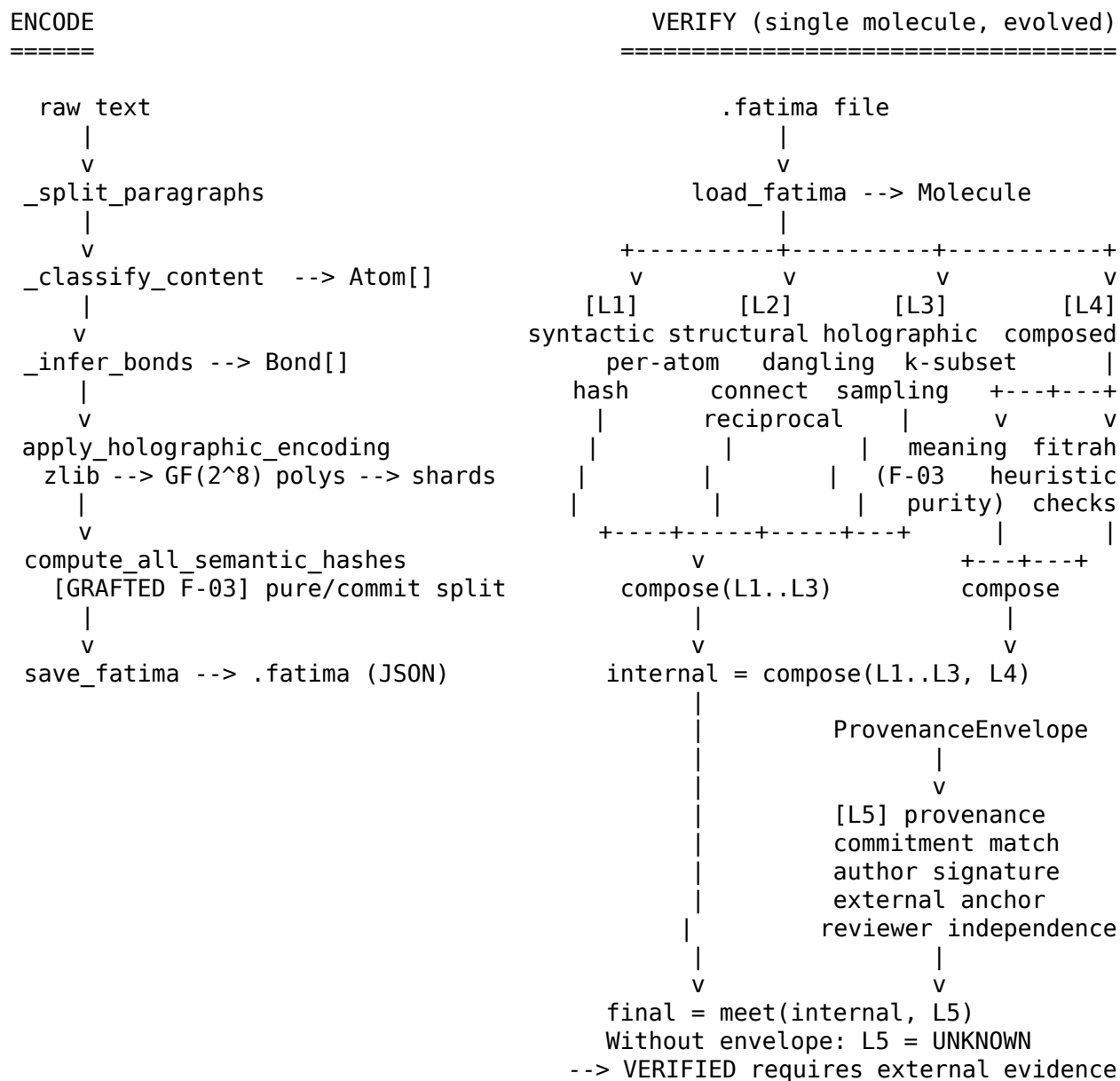
## Adversarial audit register - v0.2 prototype (forward-looking)

| ID       | Severity | Finding                                      | Proposed Repair |
|----------|----------|----------------------------------------------|-----------------|
| F-V02-01 | HIGH     | TrustStore provenance is unenforced          | R-              |
| 1:       |          | Declare trust store boundary                 |                 |
| F-V02-02 | HIGH     | Attestation has no declared criterion        | R-              |
| 2:       |          | Extend SemanticAttestation                   |                 |
| F-V02-03 | MEDIUM   | Nominal independence, not effective          | R-              |
| 3:       |          | AnchorVerifier protocol                      |                 |
| F-V02-04 | MEDIUM   | No revocation, cryptoperiod, or re-anchoring | R-4: Crypto-    |
|          |          | agility fields                               |                 |
| F-V02-05 | MEDIUM   | anchor_ref is opaque/unverified              | R-              |
| 3:       |          | AnchorVerifier protocol                      |                 |
| F-V02-06 | MEDIUM   | Purity assertion narrower than the claim     | R-              |
| 4:       |          | Widen purity tuple                           |                 |

F-V02-07 MEDIUM L2 largely redundant with construction checks R-7: Strengthen or rename L2  
 F-V02-08 LOW Share accepts unvalidated inputs R-5: Share.\_\_post\_init\_\_  
 F-V02-09 LOW bond\_type is a free string with no vocabulary R-6: Bond-type schema registry  
 F-V02-10 INFO No structured evidence in attestation R-2 (extended)  
 F-V02-11 INFO Migration module is a stub R-8: Rename stub

None have been applied. These target the provenance and attestation subsystems.

## The anatomy in one view - the two pipelines



**Field constraints that shape the whole skeleton:** GF(2^8) => <= 255 evaluation points

=> MAX\_ATOMS\_PER\_SUB = 200 => the composite skeleton exists => composite verification exists. Every large-document feature is a child of one finite-field limit.

**Honest boundary:** The architecture after today's grafts can detect meaning-integrity violations and can verify external provenance when an envelope is supplied. It cannot enforce that the trust store is independent (authority regress), cannot enforce effective reviewer independence (only nominal key separation), and cannot verify the content of a semantic attestation (only that one was signed). These are declared limits, not hidden ones.

---

*Wa ma tawfiqi illa billah. Extracted, not imagined: every symbol above appears in the source at commit dfb6205.*